

Roteiro de laboratório: particionamento e carga iterativa do arquivo ABCFarma em lotes de 10 mil registros

1. Objetivo

Neste laboratório, o aluno vai:

- diagnosticar e preparar o arquivo CSV ABCFarma;
- dividir o arquivo em vários arquivos menores, com 10 mil registros cada;
- carregar esses arquivos de forma iterativa em uma tabela única de staging;
- controlar a execução via uma tabela de log do banco;
- normalizar os dados e realizar auditoria de variação de preços;
- automatizar o processo com uma stored procedure que percorre todos os arquivos.

O cenário é inspirado no roteiro de auditoria ABCFarma, mas agora com um desafio adicional: o arquivo original é grande e precisa ser particionado antes da carga em massa.

2. Arquivos envolvidos

Os arquivos de entrada normalmente usados neste laboratório são:

- `p50_abcfarma.csv` — origem original em UTF-16
 - `p50_abcfarma_utf8.csv` — cópia convertida em UTF-8
 - arquivos particionados, como:
 - `abcfarma_0001.csv`
 - `abcfarma_0002.csv`
 - `abcfarma_0003.csv`
 - ...
-

3. Diagnóstico inicial do arquivo

Antes de qualquer carga, é importante verificar a codificação e a estrutura do arquivo. Em muitos cenários, o arquivo sai do sistema de origem em UTF-16, o que pode gerar falhas de leitura e de **BULK INSERT**.

3.1 Verificar a codificação

```
$arquivoOriginal = 'C:\dados\abcfarma\p50_abcfarma.csv'
$bytes = [System.IO.File]::ReadAllBytes($arquivoOriginal)
$bytes[0..3]
```

Se a saída for algo como:

```
255
254
...
```

significa que o arquivo está em UTF-16 LE (BOM **FF FE**).

3.2 Converter para UTF-8

```
$in = 'C:\dados\abcfarma\p50_abcfarma.csv'
$out = 'C:\dados\abcfarma\p50_abcfarma_utf8.csv'

$text = [System.IO.File]::ReadAllText($in, [System.Text.Encoding]::Unicode)
[System.IO.File]::WriteAllText($out, $text,
[System.Text.UTF8Encoding]::new($false))
```

3.3 Confirmar cabeçalho e quantidade de colunas

```
$arquivo = 'C:\dados\abcfarma\p50_abcfarma_utf8.csv'
$linhaCabecalho = Get-Content $arquivo -TotalCount 1
$campos = ($linhaCabecalho -split '\|')
```

```
$campos.Count  
$campos
```

3.4 Contar registros do arquivo

```
$arquivo = 'C:\dados\abcfarma\p50_abcfarma_utf8.csv'  
$registros = (Get-Content $arquivo).Count - 1  
Write-Host "Total de registros: $registros"
```

3.5 Visualizar as primeiras linhas

```
Get-Content 'C:\dados\abcfarma\p50_abcfarma_utf8.csv' -TotalCount 11
```

Observações

- o separador do CSV é `;`;
- o arquivo possui cabeçalho e deve ser ignorado durante a carga;
- valores monetários usam `,` como separador decimal;
- a conversão para UTF-8 é essencial para manter a leitura estável no SQL Server.

4. Divisão do arquivo em partes de 10 mil registros

Depois de validar o arquivo, o próximo passo é dividir o conteúdo em arquivos menores, cada um com 10 mil registros, para facilitar a carga incremental e a conferência.

4.1 Estrutura de saída

Crie a pasta:

```
New-Item -ItemType Directory -Force -Path 'C:\dados\abcfarma\partes'
```

4.2 Script em PowerShell para particionar o CSV

```
$arquivoEntrada = 'C:\dados\abcfarma\p50_abcfarma_utf8.csv'
$pastaSaida     = 'C:\dados\abcfarma\partes'
$batchSize      = 10000

$header = Get-Content -Path $arquivoEntrada -TotalCount 1
$linhas = Get-Content -Path $arquivoEntrada | Select-Object -Skip 1

$contador = 0
$arquivoAtual = 0
$buffer = New-Object System.Collections.Generic.List[string]

foreach ($linha in $linhas) {
    $buffer.Add($linha)
    $contador++

    if ($contador -eq $batchSize) {
        $arquivoAtual++
        $nomeArquivo = Join-Path $pastaSaida ("abcfarma_{0:0000}.csv" -f
$arquivoAtual)

        $header + $buffer | Set-Content -Path $nomeArquivo -Encoding UTF8

        Write-Host "Arquivo gerado: $nomeArquivo"
        $buffer.Clear()
        $contador = 0
    }
}

if ($buffer.Count -gt 0) {
    $arquivoAtual++
    $nomeArquivo = Join-Path $pastaSaida ("abcfarma_{0:0000}.csv" -f $arquivoAtual)
    $header + $buffer | Set-Content -Path $nomeArquivo -Encoding UTF8
    Write-Host "Arquivo final gerado: $nomeArquivo"
}

Write-Host "Total de arquivos gerados: $arquivoAtual"
```

4.3 Validar a criação dos arquivos

```
Get-ChildItem 'C:\dados\abcfarma\partes' | Select-Object Name, Length
```

4.4 Verificar o tamanho de cada arquivo

```
Get-ChildItem 'C:\dados\abcfarma\partes\*.csv' | ForEach-Object {  
    $conteudo = Get-Content $_.FullName  
    $linhas = $conteudo.Count - 1  
    Write-Host ($_.Name + " => " + $linhas + " registros")  
}
```

Objetivo didático

A ideia é que cada arquivo gerado tenha aproximadamente 10 mil linhas de dados, além da linha de cabeçalho. Isso facilita a carga incremental, a etapa de auditoria e a análise por lote.

Script auxiliar em Python

Para automatizar essa etapa, o repositório agora inclui um script que lê o CSV, preserva o cabeçalho em cada arquivo e grava os registros em lotes sequenciais:

```
python split_csv_em_lotes.py "C:\dados\abcfarma\p50_abcfarma_utf8.csv"  
"C:\dados\abcfarma\partes"
```

O script aceita o delimitador do arquivo (por padrão , , mas pode ser ajustado para |) e gera arquivos com nomes como **abcfarma_0001.csv**.

5. Preparação do banco de dados

Agora o aluno deve criar os objetos necessários para receber os arquivos particionados.

5.1 Criar schema de staging

```
CREATE SCHEMA staging;
```

5.2 Criar tabela de staging para receber um arquivo CSV

```
CREATE TABLE staging.abcfarma_raw (
  CODIGO NVARCHAR(200) NULL,
  EAN NVARCHAR(200) NULL,
  REGISTRO_ANVISA NVARCHAR(200) NULL,
  NCM NVARCHAR(50) NULL,
  CEST NVARCHAR(50) NULL,
  NOME NVARCHAR(500) NULL,
  LABORATORIO NVARCHAR(200) NULL,
  TIPO NVARCHAR(50) NULL,
  CLASSE_TERAPEUTICA NVARCHAR(500) NULL,
  COMPOSICAO NVARCHAR(1000) NULL,
  TARJA NVARCHAR(50) NULL,
  UNIDADE_DE_VENDA NVARCHAR(100) NULL,
  CNPJ_LABORATORIO NVARCHAR(50) NULL,
  APRESENTACAO NVARCHAR(200) NULL,
  LISTAPISC_OFINS NVARCHAR(50) NULL,
  RESTRHOSP NVARCHAR(50) NULL,
  CNPJ_DECLARANTE NVARCHAR(50) NULL,
  NOME_DECLARANTE NVARCHAR(500) NULL,
  PICMS NVARCHAR(50) NULL,
  VPF NVARCHAR(50) NULL,
  VPMC NVARCHAR(50) NULL,
  VPMCEMBFRAC NVARCHAR(50) NULL,
  VPMCEMBMULT NVARCHAR(50) NULL,
  CODIGO_LABORATORIO NVARCHAR(50) NULL,
  LABORATORIO_2 NVARCHAR(200) NULL,
  DESCRICAO NVARCHAR(1000) NULL,
  COMPOSICAO_2 NVARCHAR(1000) NULL,
  REGIME_PRECO NVARCHAR(100) NULL,
  LISTA_LCCT NVARCHAR(200) NULL,
  PF_20 NVARCHAR(50) NULL,
  PMC_20 NVARCHAR(50) NULL,
  FRA_20 NVARCHAR(50) NULL,
  DATA_VIGENCIA NVARCHAR(50) NULL,
  NOVO NVARCHAR(50) NULL,
  VARIACAO_PRECO NVARCHAR(200) NULL,
  REFERENCIA NVARCHAR(200) NULL,
  ATC NVARCHAR(200) NULL,
  CLASSE_TERAPEUTICA_2 NVARCHAR(500) NULL,
  PORTARIA_344_98 NVARCHAR(100) NULL,
  TISS_TUSS NVARCHAR(100) NULL,
  CONFAZ_87 NVARCHAR(100) NULL,
  CAP NVARCHAR(100) NULL,
  GGREM NVARCHAR(100) NULL,
  DCB NVARCHAR(100) NULL,
  CAS NVARCHAR(100) NULL,
  ARQUIVO_ORIGEM NVARCHAR(200) NULL,
```

```
DATA_REFERENCIA NVARCHAR(50) NULL,  
ANO_PASTA NVARCHAR(20) NULL,  
ORIGEM_ARQUIVO NVARCHAR(50) NULL  
);  
GO
```

5.3 Criar tabela de controle de carga

Essa tabela registra quantos arquivos foram processados e quantos registros foram inseridos em cada execução.

```
CREATE TABLE dbo.tb_log_carga_abcfarma (  
    IdLog INT IDENTITY(1,1) PRIMARY KEY,  
    NomeArquivo NVARCHAR(500) NOT NULL,  
    DataProcessamento DATETIME NOT NULL DEFAULT GETDATE(),  
    QtdeRegistrosInseridos INT NOT NULL,  
    StatusProcesso NVARCHAR(50) NOT NULL,  
    Observacao NVARCHAR(500) NULL  
);  
GO
```

5.4 Criar tabela final normalizada

```
CREATE TABLE dbo.abcfarma_normalizado (  
    Id BIGINT IDENTITY(1,1) PRIMARY KEY,  
    CODIGO INT NULL,  
    EAN NVARCHAR(30) NULL,  
    REGISTRO_ANVISA NVARCHAR(30) NULL,  
    NOME NVARCHAR(500) NULL,  
    LABORATORIO NVARCHAR(200) NULL,  
    CLASSE_TERAPEUTICA NVARCHAR(500) NULL,  
    COMPOSICAO NVARCHAR(1000) NULL,  
    TARJA NVARCHAR(50) NULL,  
    VPF DECIMAL(18,4) NULL,  
    VPMC DECIMAL(18,4) NULL,  
    DATA_REFERENCIA DATE NULL,  
    ARQUIVO_ORIGEM NVARCHAR(200) NULL,  
    ORIGEM_ARQUIVO NVARCHAR(50) NULL  
);  
GO
```

5.5 Criar tabela de auditoria

```
CREATE TABLE dbo.auditoria_abcfarma (  
    Id BIGINT IDENTITY(1,1) PRIMARY KEY,  
    CODIGO INT NULL,  
    LABORATORIO NVARCHAR(200) NULL,  
    DATA_REFERENCIA DATE NULL,  
    VPF DECIMAL(18,4) NULL,  
    VPMC DECIMAL(18,4) NULL,  
    VPF_ANTERIOR DECIMAL(18,4) NULL,  
    VPMC_ANTERIOR DECIMAL(18,4) NULL,  
    VARIACAO_VPF DECIMAL(18,4) NULL,  
    VARIACAO_VPMC DECIMAL(18,4) NULL,  
    FLAG_DIVERGENCIA BIT NULL  
);  
GO
```

6. Carga inicial de um único arquivo

Antes de automatizar a carga dos arquivos particionados, o aluno deve testar a carga de um único lote.

6.1 Comando de bulk insert de um único arquivo

```
BULK INSERT staging.abcfarma_raw  
FROM 'C:\dados\abcfarma\partes\abcfarma_0001.csv'  
WITH (  
    FIELDTERMINATOR = '|',  
    ROWTERMINATOR = '\n',  
    FIRSTROW = 2,  
    DATAFILETYPE = 'char',  
    CODEPAGE = '65001',  
    TABLOCK,  
    MAXERRORS = 100  
);
```

6.2 Transformação para a tabela normalizada

```
INSERT INTO dbo.abcfarma_normalizado (  
    CODIGO,  
    EAN,  
    REGISTRO_ANVISA,
```



```

NOME,
LABORATORIO,
CLASSE_TERAPEUTICA,
COMPOSICAO,
TARJA,
VPF,
VPMC,
DATA_REFERENCIA,
ARQUIVO_ORIGEM,
ORIGEM_ARQUIVO
)
SELECT
    TRY_CONVERT(INT, LTRIM(RTRIM(CODIGO))) AS CODIGO,
    LTRIM(RTRIM(EAN)) AS EAN,
    LTRIM(RTRIM(REGISTRO_ANVISA)) AS REGISTRO_ANVISA,
    LTRIM(RTRIM(NOME)) AS NOME,
    LTRIM(RTRIM(LABORATORIO)) AS LABORATORIO,
    LTRIM(RTRIM(CLASSE_TERAPEUTICA)) AS CLASSE_TERAPEUTICA,
    LTRIM(RTRIM(COMPOSICAO)) AS COMPOSICAO,
    LTRIM(RTRIM(TARJA)) AS TARJA,
    TRY_CONVERT(DECIMAL(18,4), REPLACE(REPLACE(VPF, '.', ''), ',', '.')) AS VPF,
    TRY_CONVERT(DECIMAL(18,4), REPLACE(REPLACE(VPMC, '.', ''), ',', '.')) AS VPMC,
    TRY_CONVERT(DATE, DATA_REFERENCIA) AS DATA_REFERENCIA,
    'abcfarma_0001.csv' AS ARQUIVO_ORIGEM,
    'ABCFarma' AS ORIGEM_ARQUIVO
FROM staging.abcfarma_raw
WHERE LTRIM(RTRIM(CODIGO)) IS NOT NULL
      AND LTRIM(RTRIM(CODIGO)) <> '';

```

6.3 Verificar registro carregado

```

SELECT COUNT(*) AS TotalRegistros
FROM dbo.abcfarma_normalizado;

```

7. Estrutura da auditoria de variação de preços

A infraestrutura da auditoria continua a mesma lógica do roteiro anterior:

7.1 Primeira visão: preço anterior

```

CREATE OR ALTER VIEW dbo.vw_abcfarma_precos_anteriores AS
SELECT
    CODIGO,
    LABORATORIO,
    DATA_REFERENCIA,
    VPF,
    VPMC,
    LAG(VPF) OVER (
        PARTITION BY CODIGO, LABORATORIO
        ORDER BY DATA_REFERENCIA
    ) AS VPF_ANTERIOR,
    LAG(VPMC) OVER (
        PARTITION BY CODIGO, LABORATORIO
        ORDER BY DATA_REFERENCIA
    ) AS VPMC_ANTERIOR
FROM dbo.abcfarma_normalizado;
GO

```

7.2 Segunda visão: variações relevantes

```

CREATE OR ALTER VIEW dbo.vw_abcfarma_variacoes AS
SELECT
    CODIGO,
    LABORATORIO,
    DATA_REFERENCIA,
    VPF,
    VPF_ANTERIOR,
    VPMC,
    VPMC_ANTERIOR,
    ISNULL(VPF - VPF_ANTERIOR, 0) AS VARIACAO_VPF,
    ISNULL(VPMC - VPMC_ANTERIOR, 0) AS VARIACAO_VPMC
FROM dbo.vw_abcfarma_precos_anteriores
WHERE DATA_REFERENCIA IS NOT NULL;
GO

```

7.3 Consulta de auditoria principal

```

SELECT
    CODIGO,
    LABORATORIO,
    DATA_REFERENCIA,
    VPF,
    VPF_ANTERIOR,
    VPMC,
    VPMC_ANTERIOR,
    VARIACAO_VPF,
    VARIACAO_VPMC

```

```
FROM dbo.vw_abcfarma_variacoes
WHERE ABS(VARIACAO_VPF) > 10
      OR ABS(VARIACAO_VPMC) > 10;
```

7.4 Explicação da lógica

- **PARTITION BY CODIGO, LABORATORIO** garante a comparação apenas entre itens do mesmo produto e laboratório;
- **ORDER BY DATA_REFERENCIA** define a ordem cronológica da análise;
- **LAG** pega o valor anterior para comparação;
- **ABS(...) > 10** identifica variações significativas para auditoria.

8. Passo a passo para a carga iterativa dos arquivos particionados

Agora o aluno deve carregar todos os arquivos de uma pasta em sequência, usando uma estrutura de repetição em T-SQL.

8.1 Criar tabela auxiliar para listar os arquivos da pasta

Para fins didáticos, vamos usar uma tabela temporária para guardar a lista dos arquivos da pasta de partes.

```
CREATE TABLE #Arquivos (
    IdArquivo INT IDENTITY(1,1),
    NomeArquivo NVARCHAR(500)
);
GO
```

8.2 Popular a tabela com os nomes dos arquivos

Se o ambiente permitir `xp_cmdshell`, a listagem pode ser feita diretamente no sistema operacional. Caso ele esteja desabilitado, esse passo pode ser feito em uma etapa anterior pelo banco ou pelo administrador.

```
DECLARE @Comando NVARCHAR(4000);
SET @Comando = 'dir /b "C:\dados\abcfarma\partes\*.csv"';

INSERT INTO #Arquivos (NomeArquivo)
EXEC xp_cmdshell @Comando;
```

Importante: `xp_cmdshell` deve ser habilitado pelo DBA do ambiente e usado com cuidado. Em ambientes mais restritivos, o passo pode ser substituído pelo carregamento da lista de arquivos por uma tabela externa.

8.3 Filtrar arquivos válidos

```
DELETE FROM #Arquivos
WHERE NomeArquivo IS NULL
      OR LTRIM(RTRIM(NomeArquivo)) = ''
      OR NomeArquivo LIKE '%File Not Found%';
```

8.4 Fazer o controle de loop

Agora será usada a estrutura repetitiva `WHILE` para percorrer cada arquivo da tabela.

```
DECLARE @IdAtual INT = 1;
DECLARE @NomeArquivo NVARCHAR(500);
DECLARE @CaminhoCompleto NVARCHAR(1000);
DECLARE @QtdeRegistros INT;

WHILE EXISTS (SELECT 1 FROM #Arquivos WHERE IdArquivo = @IdAtual)
BEGIN
    SELECT @NomeArquivo = NomeArquivo
    FROM #Arquivos
    WHERE IdArquivo = @IdAtual;

    SET @CaminhoCompleto = 'C:\dados\abcfarma\partes\' + @NomeArquivo;

    TRUNCATE TABLE staging.abcfarma_raw;

    BULK INSERT staging.abcfarma_raw
    FROM @CaminhoCompleto
    WITH (
        FIELDTERMINATOR = '|',
```

```

ROWTERMINATOR = '\n',
FIRSTROW = 2,
DATAFILETYPE = 'char',
CODEPAGE = '65001',
TABLOCK,
MAXERRORS = 100
);

```

```

INSERT INTO dbo.abcfarma_normalizado (
    CODIGO,
    EAN,
    REGISTRO_ANVISA,
    NOME,
    LABORATORIO,
    CLASSE_TERAPEUTICA,
    COMPOSICAO,
    TARJA,
    VPF,
    VPMC,
    DATA_REFERENCIA,
    ARQUIVO_ORIGEM,
    ORIGEM_ARQUIVO
)

```

```

SELECT

```

```

    TRY_CONVERT(INT, LTRIM(RTRIM(CODIGO))) AS CODIGO,
    LTRIM(RTRIM(EAN)) AS EAN,
    LTRIM(RTRIM(REGISTRO_ANVISA)) AS REGISTRO_ANVISA,
    LTRIM(RTRIM(NOME)) AS NOME,
    LTRIM(RTRIM(LABORATORIO)) AS LABORATORIO,
    LTRIM(RTRIM(CLASSE_TERAPEUTICA)) AS CLASSE_TERAPEUTICA,
    LTRIM(RTRIM(COMPOSICAO)) AS COMPOSICAO,
    LTRIM(RTRIM(TARJA)) AS TARJA,
    TRY_CONVERT(DECIMAL(18,4), REPLACE(REPLACE(VPF, '.', ''), ',', '.')) AS

```

VPF,

```

    TRY_CONVERT(DECIMAL(18,4), REPLACE(REPLACE(VPMC, '.', ''), ',', '.')) AS

```

VPMC,

```

    TRY_CONVERT(DATE, DATA_REFERENCIA) AS DATA_REFERENCIA,
    @NomeArquivo AS ARQUIVO_ORIGEM,
    'ABCFarma' AS ORIGEM_ARQUIVO

```

```

FROM staging.abcfarma_raw

```

```

WHERE LTRIM(RTRIM(CODIGO)) IS NOT NULL
AND LTRIM(RTRIM(CODIGO)) <> '';

```

```

SET @QtdeRegistros = @@ROWCOUNT;

```

```

INSERT INTO dbo.tb_log_carga_abcfarma (
    NomeArquivo,
    QtdeRegistrosInseridos,
    StatusProcesso,
    Observacao
)

```

```

VALUES (
    @NomeArquivo,
    @QtdeRegistros,
    'OK',
    'Arquivo carregado com sucesso.'
)

```

```

);

```

```
SET @IdAtual = @IdAtual + 1;  
END;
```

8.5 Verificar log de execução

```
SELECT  
    IdLog,  
    NomeArquivo,  
    DataProcessamento,  
    QtdeRegistrosInseridos,  
    StatusProcesso,  
    Observacao  
FROM dbo.tb_log_carga_abcfarma  
ORDER BY IdLog;
```

8.6 Verificar total final carregado

```
SELECT COUNT(*) AS TotalRegistrosNaTabelaFinal  
FROM dbo.abcfarma_normalizado;
```

8.7 Verificar total de arquivos processados

```
SELECT COUNT(*) AS TotalArquivosProcessados  
FROM dbo.tb_log_carga_abcfarma  
WHERE StatusProcesso = 'OK';
```

9. Implementação de uma stored procedure para automatizar a carga em lote

A seguir, o aluno prepara uma stored procedure que encapsula a lógica dos arquivos particionados e faz o controle final.

```

CREATE OR ALTER PROCEDURE dbo.usp_carga_lotes_abcfarma
    @PastaArquivos NVARCHAR(500)
AS
BEGIN
    SET NOCOUNT ON;

    CREATE TABLE #Arquivos (
        IdArquivo INT IDENTITY(1,1),
        NomeArquivo NVARCHAR(500)
    );

    DECLARE @Comando NVARCHAR(4000);
    SET @Comando = 'dir /b "' + @PastaArquivos + '\\*.csv"';

    INSERT INTO #Arquivos (NomeArquivo)
    EXEC xp_cmdshell @Comando;

    DELETE FROM #Arquivos
    WHERE NomeArquivo IS NULL
        OR LTRIM(RTRIM(NomeArquivo)) = ''
        OR NomeArquivo LIKE '%File Not Found%';

    DECLARE @IdAtual INT = 1;
    DECLARE @NomeArquivo NVARCHAR(500);
    DECLARE @CaminhoCompleto NVARCHAR(1000);
    DECLARE @QtdeRegistros INT;
    DECLARE @TotalArquivos INT = 0;
    DECLARE @TotalRegistros INT = 0;

    WHILE EXISTS (SELECT 1 FROM #Arquivos WHERE IdArquivo = @IdAtual)
    BEGIN
        SELECT @NomeArquivo = NomeArquivo
        FROM #Arquivos
        WHERE IdArquivo = @IdAtual;

        SET @CaminhoCompleto = @PastaArquivos + '\\\' + @NomeArquivo;

        BEGIN TRY
            TRUNCATE TABLE staging.abcfarma_raw;

            BULK INSERT staging.abcfarma_raw
            FROM @CaminhoCompleto
            WITH (
                FIELDTERMINATOR = '|',
                ROWTERMINATOR = '\n',
                FIRSTROW = 2,
                DATAFILETYPE = 'char',
                CODEPAGE = '65001',
                TABLOCK,
                MAXERRORS = 100
            );

            INSERT INTO dbo.abcfarma_normalizado (
                CODIGO,
                EAN,
                REGISTRO_ANVISA,
                NOME,
            
```

```

LABORATORIO,
CLASSE_TERAPEUTICA,
COMPOSICAO,
TARJA,
VPF,
VPMC,
DATA_REFERENCIA,
ARQUIVO_ORIGEM,
ORIGEM_ARQUIVO
)
SELECT
    TRY_CONVERT(INT, LTRIM(RTRIM(CODIGO))) AS CODIGO,
    LTRIM(RTRIM(EAN)) AS EAN,
    LTRIM(RTRIM(REGISTRO_ANVISA)) AS REGISTRO_ANVISA,
    LTRIM(RTRIM(NOME)) AS NOME,
    LTRIM(RTRIM(LABORATORIO)) AS LABORATORIO,
    LTRIM(RTRIM(CLASSE_TERAPEUTICA)) AS CLASSE_TERAPEUTICA,
    LTRIM(RTRIM(COMPOSICAO)) AS COMPOSICAO,
    LTRIM(RTRIM(TARJA)) AS TARJA,
    TRY_CONVERT(DECIMAL(18,4), REPLACE(REPLACE(VPF, '.', ''), ',', '')) AS VPF,
    TRY_CONVERT(DECIMAL(18,4), REPLACE(REPLACE(VPMC, '.', ''), ',', '')) AS VPMC,
    TRY_CONVERT(DATE, DATA_REFERENCIA) AS DATA_REFERENCIA,
    @NomeArquivo AS ARQUIVO_ORIGEM,
    'ABCFarma' AS ORIGEM_ARQUIVO
FROM staging.abcfarma_raw
WHERE LTRIM(RTRIM(CODIGO)) IS NOT NULL
      AND LTRIM(RTRIM(CODIGO)) <> '';

SET @QtdeRegistros = @@ROWCOUNT;
SET @TotalRegistros = @TotalRegistros + @QtdeRegistros;
SET @TotalArquivos = @TotalArquivos + 1;

INSERT INTO dbo.tb_log_carga_abcfarma (
    NomeArquivo,
    QtdeRegistrosInseridos,
    StatusProcesso,
    Observacao
)
VALUES (
    @NomeArquivo,
    @QtdeRegistros,
    'OK',
    'Arquivo carregado com sucesso.'
);

END TRY
BEGIN CATCH
    INSERT INTO dbo.tb_log_carga_abcfarma (
        NomeArquivo,
        QtdeRegistrosInseridos,
        StatusProcesso,
        Observacao
    )
    VALUES (
        @NomeArquivo,
        0,
        'ERRO',

```



```

        ERROR_MESSAGE()
    );
END CATCH

SET @IdAtual = @IdAtual + 1;
END;

SELECT
    @TotalArquivos AS TotalArquivosProcessados,
    @TotalRegistros AS TotalRegistrosInseridos;
END;
GO

```

9.1 Execução da stored procedure

```

EXEC dbo.usp_carga_lotes_abcfarma
    @PastaArquivos = 'C:\dados\abcfarma\partes';

```

9.2 Consultar resultado final

```

SELECT
    SUM(QtdeRegistrosInseridos) AS TotalTodosRegistros,
    COUNT(*) AS TotalArquivosProcessados
FROM dbo.tb_log_carga_abcfarma
WHERE StatusProcesso = 'OK';

```

10. Sistema de controle e rastreabilidade

O banco precisa manter controle sobre cada lote processado para permitir:

- auditoria de execução;
- reproprocessamento em caso de falha;
- contabilização de lotes e registros carregados;
- investigação de anomalias por arquivo;
- rastreio da etapa de carga.

10.1 Consulta para ver a evolução dos arquivos

```
SELECT
    NomeArquivo,
    DataProcessamento,
    QtdeRegistrosInseridos,
    StatusProcesso,
    Observacao
FROM dbo.tb_log_carga_abcfarma
ORDER BY DataProcessamento;
```

10.2 Consulta resumida por status

```
SELECT
    StatusProcesso,
    COUNT(*) AS QuantidadeArquivos,
    SUM(QtdeRegistrosInseridos) AS TotalRegistros
FROM dbo.tb_log_carga_abcfarma
GROUP BY StatusProcesso;
```

10.3 Consulta para verificar o volume do banco final

```
SELECT COUNT(*) AS TotalRegistrosNaTabelaFinal
FROM dbo.abcfarma_normalizado;
```

11. Checklist do laboratório

O aluno deve concluir as etapas a seguir:

- ☐ verificar o encoding do arquivo original;
- ☐ converter para UTF-8 quando necessário;
- ☐ validar o delimitador |;
- ☐ revisar cabeçalho e quantidade de colunas;

- ☐ dividir o CSV em partes de 10 mil registros;
 - ☐ criar schema e tabelas de staging;
 - ☐ criar tabela de controle para log da carga;
 - ☐ criar stored procedure de iteração;
 - ☐ executar a carga dos arquivos em sequência;
 - ☐ validar a quantidade de registros inseridos;
 - ☐ calcular a auditoria de variação de preços;
 - ☐ confirmar o total final de registros carregados.
-

12. Conclusão

Este laboratório simula um cenário real de carga de dados fiscais em lote. O aluno aprende a separar um arquivo grande em blocos menores, carregar cada bloco em sequência, registrar a execução em tabela de controle e, por fim, usar a base normalizada para auditoria de preços.

O ponto central do exercício não é apenas a tecnologia de **BULK INSERT**, mas também a disciplina de governança de dados: particionar o arquivo, controlar a execução, registrar os dados e manter rastreabilidade do processo.

Esse tipo de abordagem é muito comum em ambientes corporativos, principalmente quando a base de entrada é muito grande e a auditoria precisa ser feita de forma segura e documentada.